

LAPORAN TUGAS BESAR 1
IF3270 PEMBELAJARAN MESIN
FEEDFORWARD NEURAL NETWORK



Disusun oleh Kelompok 49:

13523074	Ahsan Malik Al Farisi
13523113	Kefas Kurnia Jonathan
13622076	Ziyan Agil Nur Ramadhan

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

1 Deskripsi Persoalan.....	3
2 Hasil dan Pembahasan.....	8
2.1 Implementasi.....	8
2.1.1 Kelas, Atribut, dan Metode.....	8
2.1.2 Forward Propagation.....	13
2.1.3 Backward Propagation dan Weight Update.....	13
2.2 Hasil Pengujian.....	14
2.2.1 Pengaruh Depth dan Width.....	14
2.2.2 Pengaruh Fungsi Aktivasi.....	17
2.2.3 Pengaruh Learning Rate.....	22
2.2.4 Pengaruh Inisialisasi Bobot.....	26
2.2.5 Pengaruh Regularisasi.....	30
2.2.6 Pengaruh Normalisasi RMSNorm.....	34
2.2.7 Perbandingan Metode Optimizer SGD dan Adam.....	36
2.2.8 Perbandingan dengan Library Sklearn.....	37
3 Kesimpulan dan Saran.....	39
3.1 Kesimpulan.....	39
3.2 Saran.....	40
REFERENSI.....	41
LAMPIRAN.....	42
A. Pembagian Tugas.....	42
B. Form Penggunaan AI.....	42

Daftar Tabel

Tabel 1.1 Deskripsi Fitur pada Dataset.....	3
Tabel 1.2 Definisi tiap Fungsi Aktivasi.....	4
Tabel 1.3 Definisi tiap Loss Function.....	4
Tabel 1.4 Turunan Pertama untuk Tiap Fungsi Aktivasi.....	6
Tabel 1.5 Turunan Pertama untuk Setiap Fungsi Loss terhadap Bobot suatu FFNN.....	7
Tabel 2.1.1.1 Kelas, Atribut, dan Metode.....	8
Tabel 2.2.1.1 Hasil Metrik Pengujian Depth dan Width.....	14
Tabel 2.2.1.2 Hasil Grafik Loss Pengujian Depth dan Width.....	15
Tabel 2.2.2.1 Hasil Metrik Pengujian Fungsi Aktivasi.....	17
Tabel 2.2.2.2 Hasil Grafik Loss Pengujian Fungsi Aktivasi.....	18
Tabel 2.2.2.3 Hasil Grafik Distribusi Weight Pengujian Fungsi Aktivasi.....	20
Tabel 2.2.2.4 Hasil Grafik Distribusi Gradien Pengujian Fungsi Aktivasi.....	21
Tabel 2.2.3.1 Hasil Metrik Pengujian Nilai Learning Rate.....	22
Tabel 2.2.3.2 Hasil Grafik Loss Pengujian Nilai Learning Rate.....	23
Tabel 2.2.3.3 Hasil Grafik Distribusi Weight Pengujian Nilai Learning Rate.....	24
Tabel 2.2.3.4 Hasil Grafik Distribusi Gradien Pengujian Nilai Learning Rate.....	25
Tabel 2.2.4.1 Hasil Metrik Pengujian Metode Inisialisasi Bobot.....	26
Tabel 2.2.4.2 Hasil Grafik Loss Pengujian Metode Inisialisasi Bobot.....	26
Tabel 2.2.4.3 Hasil Grafik Distribusi Weight Pengujian Metode Inisialisasi Bobot.....	29
Tabel 2.2.4.4 Hasil Grafik Distribusi Gradien Pengujian Metode Inisialisasi Bobot.....	30
Tabel 2.2.5.1 Hasil Metrik Weight Pengujian Regularisasi.....	31
Tabel 2.2.5.2 Hasil Grafik Loss Pengujian Regularisasi.....	31
Tabel 2.2.5.3 Hasil Grafik Distribusi Weight Pengujian Regularisasi.....	32
Tabel 2.2.5.4 Hasil Grafik Distribusi Gradient Pengujian Regularisasi.....	33
Tabel 2.2.6.1 Hasil Metrik Pengujian Normalisasi.....	34
Tabel 2.2.6.2 Hasil Metrik Grafik Loss Pengujian Normalisasi.....	34
Tabel 2.2.6.3 Hasil Grafik Distribusi Weight Pengujian Normalisasi.....	35
Tabel 2.2.6.4 Hasil Grafik Distribusi Gradien Pengujian Normalisasi.....	36
Tabel 2.2.7.1 Hasil Grafik Loss Pengujian Metode Optimizer.....	36
Tabel 2.2.8.1 Hasil Metrik Pengujian Perbandingan dengan Library Sklearn.....	37

1 Deskripsi Persoalan

Sebuah perusahaan berencana mengembangkan sistem evaluasi kredit otomatis untuk meningkatkan efektivitas dalam proses persetujuan pinjaman. Setelah melakukan analisis terhadap data historis, perusahaan menyimpulkan bahwa pola karakteristik pemohon pinjaman di periode mendatang cenderung serupa dengan pola pada data sebelumnya.

Dengan memanfaatkan informasi yang tersedia, perusahaan ingin membangun model yang mampu memprediksi status persetujuan pinjaman untuk setiap individu, sehingga proses pengambilan keputusan dapat dilakukan dengan lebih cepat.

Pada tugas besar ini, penulis akan melakukan analisis dan pemodelan menggunakan dataset Global Student Placement & Salary Dataset. Dataset ini berisi kumpulan data global mengenai performa akademik mahasiswa serta outcome penempatan kerja dan gaji setelah lulus. Dataset yang akan digunakan dapat diakses [di sini](#). Berikut deskripsi dari tiap fitur yang ada pada dataset yang akan digunakan.

Tabel 1.1 Deskripsi Fitur pada Dataset

Column	Description	Type
cgpa	Cumulative Grade Point Average (skala 4.0–10.0)	Float
backlogs	Jumlah mata kuliah yang belum lulus	Integer
college_tier	Tingkatan institusi	Categorical
country	Negara kelulusan	Categorical
university_ranking_band	Kategori peringkat global universitas	Categorical
internship_count	Jumlah internship yang telah diselesaikan	Integer
aptitude_score	Skor tes kemampuan kuantitatif	Integer
communication_score	Skor kemampuan komunikasi	Integer
specialization	Bidang akademik	Categorical
industry	Industri tempat bekerja	Categorical
internship_quality_score	Skor kualitas pengalaman internship	Float

placement_status	Status penempatan kerja	Categorical
-------------------------	-------------------------	-------------

Pada dataset di atas, akan diimplementasikan algoritma pembelajaran mesin yaitu Feedforward Neural Network (FFNN). Implementasi modul FFNN akan memenuhi ketentuan-ketentuan berikut.

- FFNN yang diimplementasikan dapat menerima jumlah neuron dari tiap layer (termasuk input layer dan output layer).
- FFNN yang diimplementasikan dapat menerima fungsi aktivasi dari tiap layer. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut.

Tabel 1.2 Definisi tiap Fungsi Aktivasi

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

- FFNN yang diimplementasikan dapat menerima fungsi loss dari model tersebut. Pilihan loss function yang akan diimplementasikan adalah sebagai berikut.

Tabel 1.3 Definisi tiap Loss Function

Nama Loss Function	Definisi Fungsi
MSE (Mean Squared Error)	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Binary Cross-Entropy	$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=1}^n (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$ y_i = Label biner aktual (0 atau 1) $\hat{y}_i$ = Nilai prediksi dari y_i n = Batch size
Categorical Cross-Entropy	$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C (y_{ij} \log \hat{y}_{ij})$ y_{ij} = Nilai aktual instansi i untuk kelas j $\hat{y}_{ij}$ = Nilai prediksi dari y_{ij} C = Jumlah kelas n = Batch size

Sebagai catatan, binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2. Kemudian, log yang digunakan merupakan logaritma natural (logaritma dengan basis e).

- Terdapat mekanisme untuk inialisasi bobot tiap neuron (termasuk bias). Pilihan metode inialisasi bobot yang akan diimplementasikan adalah sebagai berikut.
 - Zero initialization
 - Random dengan distribusi uniform.
 - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal).
 - Menerima parameter seed untuk reproducibility.
 - Random dengan distribusi normal.
 - Menerima parameter mean dan variance.
 - Menerima parameter seed untuk reproducibility.
- Instance model yang diinisialisasikan harus bisa menyimpan bobot tiap neuron (termasuk bias).
- Instance model yang diinisialisasikan harus bisa menyimpan gradien bobot tiap neuron (termasuk bias).
- Instance model memiliki method untuk menampilkan distribusi bobot dari tiap layer.

- Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk menampilkan distribusi gradien bobot dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk save dan load.
- Model memiliki implementasi forward propagation dengan ketentuan sebagai berikut:
 - Dapat menerima input berupa batch.
- Model memiliki implementasi backward propagation untuk menghitung perubahan gradien:
 - Dapat menangani perhitungan perubahan gradien untuk input data batch.
 - Gunakan konsep chain rule untuk menghitung gradien tiap bobot terhadap loss function.
 - Berikut merupakan turunan pertama untuk setiap fungsi aktivasi:

Tabel 1.4 Turunan Pertama untuk Tiap Fungsi Aktivasi

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$\frac{d(Linear(x))}{dx} = 1$
ReLU	$\frac{d(ReLU(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x + e^{-x}} \right)^2$

Softmax	Untuk vektor $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$: $\frac{d(\text{softmax}(\vec{x}))}{d\vec{x}} = \begin{bmatrix} \frac{\partial(\text{softmax}(\vec{x})_1)}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(\text{softmax}(\vec{x})_n)}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x})_n)}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$: $\frac{\partial(\text{softmax}(\vec{x})_i)}{\partial x_j} = \text{softmax}(\vec{x})_i (\delta_{i,j} - \text{softmax}(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$
----------------	---

- Berikut merupakan turunan pertama untuk setiap fungsi loss terhadap bobot suatu FFNN.

Tabel 1.5 Turunan Pertama untuk Setiap Fungsi Loss terhadap Bobot suatu FFNN

Nama Fungsi	Turunan Pertama Fungsi Loss
MSE (Mean Squared Error)	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial W}$
Binary Cross-Entropy	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \frac{\partial \hat{y}_i}{\partial W}$
Categorical Cross-Entropy	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{y_{ij}}{\hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W}$

2 Hasil dan Pembahasan

2.1 Implementasi

2.1.1 Kelas, Atribut, dan Metode

Tabel 2.1.1.1 Kelas, Atribut, dan Metode

Kelas	Metode	Atribut	Keterangan
class Activation	def forward(self, x)	x	Kelas abstrak untuk fungsi aktivasi. Memiliki metode def forward yang menerima x sebagai tensor.
class Linear(Activation)			Kelas untuk fungsi aktivasi linear. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi linear.
class ReLU(Activation)			Kelas untuk fungsi aktivasi ReLU. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi ReLU.
class Sigmoid(Activation)			Kelas untuk fungsi aktivasi linear. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi sigmoid.
class Tanh(Activation)			Kelas untuk fungsi aktivasi linear. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi tanh.
class Softmax(Activation)			Kelas untuk fungsi aktivasi linear. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi softmax.
class Swish(Activation)			Kelas untuk fungsi aktivasi linear. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi swish.
class LeakyReLU(Activation)			Kelas untuk fungsi aktivasi linear. Memiliki metode def forward yang menerima x sebagai tensor dan mengembalikan tensor hasil fungsi aktivasi LeakyReLU.
class (LossFunction)	def forward(self, y_true, y_pred)	y_true y_pred	Kelas abstrak untuk <i>loss function</i> . Memiliki metode def forward yang menerima y_true (label) dan y_pred (output model) sebagai tensor.
class			Kelas untuk MSE <i>loss function</i> . Memiliki

MSE (LossFunction)			metode def forward yang menerima y_true (label) dan y_pred (output model) sebagai tensor kemudian mengembalikan nilai loss dengan rumus MSE.
class BinaryCrossEntropy (LossFunction)			Kelas untuk Binary Cross Entropy <i>loss function</i> . Memiliki metode def forward yang menerima y_true (label) dan y_pred (output model) sebagai tensor kemudian mengembalikan nilai loss dengan rumus Binary Cross Entropy.
class CategoricalCrossEntropy (LossFunction)			Kelas untuk Categorical Cross Entropy <i>loss function</i> . Memiliki metode def forward yang menerima y_true (label) dan y_pred (output model) sebagai tensor kemudian mengembalikan nilai loss dengan rumus Categorical Cross Entropy.
class (Layer)	def __init__(self, input_size, output_size, initialization_type="random_uniform",)	input_size output_size initialization_type	Menginisialisasi dimensi dan bobot layer. Terdapat 5 metode inisialisasi bobot, yaitu zero, random uniform, random normal, he, dan xavier.
	def forward(self, inputs)	inputs	Melakukan operasi linear: output = inputs @ W + b, kemudian menyimpan input ke self.inputs. Metode ini mengembalikan tensor berukuran (batch × output_size).
	def get_parameters(self)	-	Mengembalikan list [self.weights, self.bias] yang digunakan oleh optimizer untuk mengakses semua parameter yang perlu diperbarui.
	-	weights	Matriks bobot W berukuran (input_size × output_size) yang diinisialisasi sesuai initialization_type.
	-	bias	Vektor bias b berukuran (1 × output_size) yang selalu diinisialisasi oleh nol.
class (Optimizer)	def __init__(self, parameters, l1_lambda=0.0, l2_lambda=0.0)	l1_lambda l2_lambda	Metode regularisasi L1 dan L2.
	def	-	Metode untuk menambahkan gradien

	<code>_apply_regularization(self)</code>		regularisasi ke p.grad sebelum update.
	<code>def step(self)</code>	-	Metode abstrak untuk memperbarui bobot.
	<code>def zero_grad(self)</code>	-	Metode untuk memanggil p.zero_grad() untuk setiap parameter yang dipanggil setelah setiap step.
class SGD(Optimizer)	<code>def __init__(self, parameters, learning_rate=0.01, l1_lambda=0.0, l2_lambda=0.0)</code>	l1_lambda l2_lambda	Menginisialisasi parameter, regularisasi L1 dan L2, serta learning rate.
	<code>def step(self)</code>	-	Metode untuk menerapkan regularisasi serta memperbarui nilai bobot (dengan Stochastic Gradient Descent) berdasarkan gradien yang telah dihitung oleh backward.
class Adam(Optimizer)	<code>def __init__(self, parameters, learning_rate=0.001, beta1=0.9, beta2=0.999, eps=1e-8, l1_lambda=0., l2_lambda=0.0,)</code>	learning_rate beta1 beta2 t m v	Menginisialisasi parameter, regularisasi L1 dan L2, learning rate, beta1, beta2, serta eps. Atribut tambahan untuk kelas ini adalah. beta 1: koefisien decay momen pertama (rata-rata gradien). Beta 2: koefisien decay momen kedua (rata-rata kuadrat gradien). t: counter langkah (timestep). Digunakan koreksi bias. m: estimasi momen pertama (mean gradien) untuk setiap parameter. v: estimasi momen kedua (variance gradien) untuk setiap parameter.
	<code>def step(self)</code>	-	Metode untuk menerapkan regularisasi serta memperbarui nilai bobot (dengan Adam Gradient Descent) berdasarkan gradien yang telah dihitung oleh backward.
class RMSNorm:	<code>def __init__(self, size, eps=1e-8)</code>	size eps gamma	Menginisialisasi gamma sebagai Tensor ones berukuran $(1 \times \text{size})$. gamma: parameter yang dapat dipelajari (skala).
	<code>def forward(self, inputs)</code>	inputs	Metode yang menghitung RMS, menerima parameter inputs berupa Tensor.
	<code>def get_parameters(self)</code>	-	Mengembalikan nilai gamma agar optimizer dapat memperbarui parameter normalisasi.

class LossFunction:	def forward(self, y_true, y_pred)	y_true, y_pred	Kelas untuk menjaga fungsi loss.
class MSE(LossFunction):			Kelas untuk menghitung Mean Squared Error dengan mengonversi y_true ke Tensor, hitung selisih, dikuadratkan kemudian dirata-ratakan dengan faktor 1/jumlah_elemen.
class BinaryCrossEntropy(LossFunction):			Kelas untuk menghitung Binary Cross Entropy dengan $\text{eps} = 1e^{-15}$.
class CategoricalCrossEntropy(LossFunction):			Kelas untuk menghitung Categorical Cross Entropy, untuk label <i>one-hot</i> atau <i>multiclass</i> .
class Tensor:	def __init__(self, data, _children=(), _op="", label="")	data, _children, _op, label	Konstruktor tensor <i>auto differentiation</i> yang menyimpan nilai numerik, gradien awal nol, <i>node parent</i> untuk graf komputasi, nama operasi, label, dan fungsi <i>backward default</i> .
	def __getstate__(self)	-	Metode untuk serialisasi, yaitu menyalin suatu objek lalu menghapus " <i>backward</i> " dan " <i>prev</i> " agar aman diserialisasi.
	def __setstate__(self, state)	-	Metode untuk deserialisasi, yaitu memulihkan state, <i>set</i> ulang self._prev menjadi set kosong dan self._backward ke fungsi no-op.
	def __repr__(self)	-	Metode representasi string untuk <i>debugging</i> .
	@staticmethod def _to_tensor(x)	x	Metode statis untuk normalisasi input.
	@staticmethod def _unbroadcast(grad, original_shape)	grad, original_shape	Metode statis untuk mengembalikan <i>shape</i> gradien hasil <i>broadcasting</i> ke shape asli tensor.
	def zero_grad(self)	-	Metode untuk me-reset gradien tensor menjadi nol.
	def __add__(self, other)	other	Metode untuk operasi penjumlahan tensor dengan graph tracking. Backward menjumlahkan gradien ke kedua operand

			dengan penyesuaian broadcasting
<code>def __radd__(self, other)</code>	other		Metode untuk penjumlahan sisi kanan, merupakan delegasi ke <code>__add__</code> .
<code>def __sub__(self, other)</code>	other		Metode untuk operasi pengurangan yang diimplementasi dengan <code>self + (-other)</code> .
<code>def __rsub__(self, other)</code>	other		Metode untuk pengurangan sisi kanan, yang diimplementasi dengan <code>other + (-self)</code>
<code>def __neg__(self)</code>	-		Metode untuk negasi unary.
<code>def __mul__(self, other)</code>	other		Metode untuk operasi perkalian <i>element-wise</i> .
<code>def __rmul__(self, other)</code>	other		Metode untuk perkalian sisi kanan, merupakan delegasi dari <code>__mul__</code> .
<code>def __pow__(self, exponent)</code>	exponent		Metode untuk pangkat skalar (int/float).
<code>def __matmul__(self, other)</code>	other		Metode untuk perkalian matriks '@'.
<code>def __rmatmul__(self, other)</code>	other		Metode untuk perkalian matriks '@' sisi kanan.
<code>def dot(self, other)</code>	other		Metode untuk dot product menggunakan <code>np.dot</code>
<code>def exp(self)</code>	-		Metode untuk fungsi eksponensial <i>element-wise</i> .
<code>def log(self)</code>	-		Metode untuk logaritma natural <i>element-wise</i> .
<code>def sum(self, axis=None, keepdims=False, **kwargs)</code>	axis, keepdims, **kwargs		Metode untuk reduksi penjumlahan.
<code>def relu(self)</code>	-		Metode untuk aktivasi ReLU <i>element-wise</i> .
<code>def backward(self, grad=None)</code>	grad		Metode untuk menjalankan <i>reverse-mode auto differentiation</i> termasuk validasi grad output, inisialisasi grad root, dan propagasi gradien dari output ke input secara terbalik.

	def build_topo(no de)		Metode rekursif lokal untuk menyusun urutan topologis <i>node graph</i> komputasi sebelum <i>backpropagation</i> .
--	-----------------------------	--	--

2.1.2 Forward Propagation

Forward propagation pada tugas besar ini diimplementasikan secara sekuensial, yaitu data *input* diproses berurutan dari layer pertama hingga layer terakhir untuk menghasilkan prediksi. Secara garis besar, alurnya terdapat ada *method forward* di *file ffn*, di mana model melakukan iterasi setiap komponen layer dan terus memperbarui nilai aktivasi hingga menjadi output akhir. Pada proses pelatihan, *forward propagation* dipanggil untuk setiap *mini-batch*, kemudian hasil prediksi tersebut digunakan untuk menghitung nilai *loss*.

Pada *linear layer*, transformasi yang digunakan adalah transformasi $Z = XW + b$. *Input* terlebih dahulu dipastikan menjadi objek Tensor, lalu dilakukan perkalian matriks antara *input* dan bobot, kemudian ditambahkan dengan bias. Dengan demikian, setiap neuron pada layer menghasilkan kombinasi linear dari fitur *input* yang akan diteruskan ke tahap berikutnya. Setelah transformasi linear, model yang dirancang juga menerapkan fungsi aktivasi agar *network* dapat mempelajari relasi *non-linear*. Pada tugas besar ini, digunakan ReLU pada *hidden layer* dengan bentuk $\text{ReLU}(x) = \max(0, x)$, sedangkan *output layer* menggunakan *softmax*. Implementasi *softmax* juga memberikan stabilitas numerik dengan mengurangi nilai maksimum per sampel sebelum eksponensial, sehingga komputasi lebih stabil.

Model yang dibuat juga menyediakan normalisasi RMSNorm sebagai komponen opsional di antara layer linear dan aktivasi. Metode ini menormalkan vektor aktivasi berdasarkan *root mean square*, lalu mengalikannya dengan parameter skala yang dapat dipelajari. RMSNorm ini membantu menjaga skala aktivasi tetap terkontrol selama *forward pass*. Selain itu, seluruh operasi pada *forward propagation* berjalan di atas struktur Tensor yang sekaligus membangun *computation graph* untuk *auto differentiation*. Output *forward* berupa probabilitas prediksi yang kemudian dievaluasi menggunakan *Categorical Cross Entropy*.

2.1.3 Backward Propagation dan Weight Update

Backward Propagation pada tugas besar ini dimulai ketika nilai *loss* untuk satu *mini-batch* selesai dihitung. Pada tahap ini, sistem tidak menghitung ulang *forward*, tetapi menelusuri *computation graph* yang sudah dibentuk dengan *autodiff* ketika *forward pass*, kemudian menyebarkan gradien dari *output* ke seluruh parameter model. Mekanisme ini membuat perhitungan turunan lebih efisien, karena setiap operasi sudah menyimpan aturan turunannya masing-masing.

Di level Tensor, proses *backward* terdapat pada *autodiff.py*. Ini diimplementasikan dengan menyusun urutan *node* dengan *topological sort* melalui fungsi lokal bernama *build_topo*, kemudian mengeksekusi propagasi gradien secara terbalik. Pendekatan seperti ini memastikan setiap *node* diproses setelah semua dependensinya selesai, sehingga gradien yang dihitung tetap konsisten terhadap aturan *chain rule*.

Kontribusi gradien untuk setiap operasi dapat dilihat pada fungsi *_backward* milik setiap operator di kelas Tensor. Sebagai contoh, penjumlahan mengakumulasi gradien dengan penyesuaian *broadcast*, perkalian matriks menghitung gradien terhadap dua operand, dan ReLU menerapkan *mask* turunan. Pola yang kami gunakan adalah akumulasi dengan operator *+=*, sehingga jika satu tensor berkontribusi kepada beberapa *computation line*, seluruh kontribusi gradien tetap terjumlah dengan benar.

Setelah seluruh gradien tersedia, proses *weight update* dijalankan dengan *optimizer.step*, kemudian gradien di-reset dengan *optimizer.zero_grad*. Parameter yang di-update berasal dari layer yang memiliki *get_parameters*, seperti bobot dan bias. Dengan demikian, tiap *mini-batch* selalu mengikuti aturan yang sama, yaitu *forward*, menghitung *loss*, *backward*, *update* parameter, dan membersihkan gradien sebelum *batch* berikutnya.

Pada implementasi kami, kami menyediakan dua *optimizer* yaitu SGD dan Adam. Untuk SGD, pembaruan parameter mengikuti bentuk dasar $\theta \leftarrow \theta - \eta \nabla \theta L$. Untuk Adam, *update* menggunakan estimasi momen pertama dan kedua gradien, dilanjutkan *bias correction*, kemudian pembaruan parameter adaptif. Selain itu, implementasi *optimizer* juga mendukung regularisasi L1 dan L2, yang jika diaktifkan gradien ditambahkan komponen penalti L2 atau L1 sebelum langkah *update*.

2.2 Hasil Pengujian

2.2.1 Pengaruh Depth dan Width

Pengujian depth dan width masing-masing dilakukan dengan 3 variasi nilai dan memiliki beberapa variabel kontrol, yaitu learning rate dengan nilai 0.01, fungsi aktivasi ReLU, dan tidak memakai nilai regularisasi L1 dan L2.

Variasi dari depth dilakukan dengan 1, 3, dan 5 kedalaman dengan tiap layer berjumlah 32 neuron. Sedangkan variasi dari width dilakukan menggunakan 2 layer dengan variasi jumlah layer senilai 16, 32, dan 64. Berikut merupakan hasil dari pengujian.

Tabel 2.2.1.1 Hasil Metrik Pengujian Depth dan Width

No.	hidden_layers	Accuracy	Precision	Recall	F1
-----	---------------	----------	-----------	--------	----

Depth					
1	[32]	0.744667	0.760116	0.854821	0.804691
2	[32, 32, 32]	0.734667	0.758112	0.835320	0.794845
3	[32, 32, 32, 32, 32]	0.734667	0.767040	0.816901	0.791186
Width					
1	[16, 16]	0.738000	0.756286	0.847237	0.799182
2	[32, 32]	0.740667	0.757722	0.850488	0.801429
3	[64, 64]	0.736000	0.758587	0.837486	0.796087

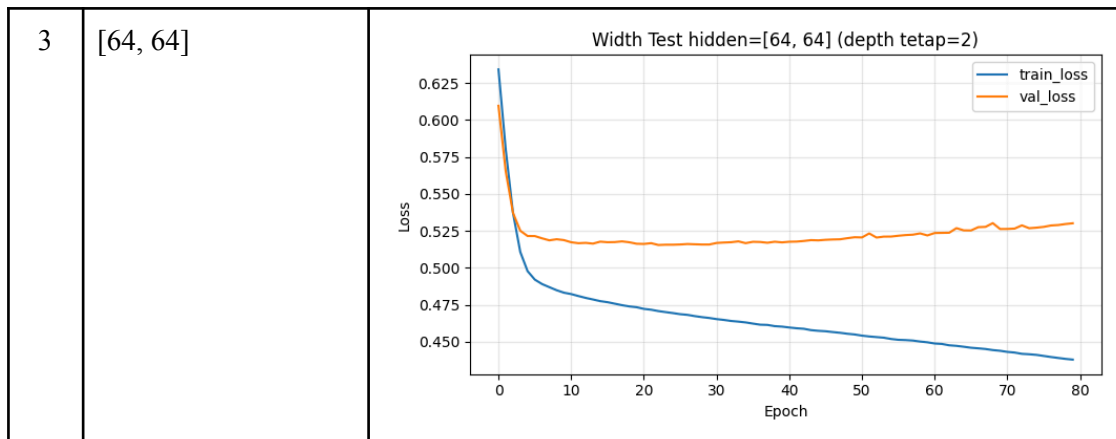
Berdasarkan Tabel 2.2.1.1, peningkatan kedalaman maupun lebar pada arsitektur model justru menurunkan performa secara keseluruhan. Pada pengujian depth, konfigurasi paling sederhana dengan 1 layer (32 neuron) memberikan hasil terbaik (akurasi 74,47%). Sementara itu, pada penambahan width (2 layer), performa model sempat naik optimal di jumlah 32 neuron (akurasi 74,06%) namun kembali turun saat neuron diperbanyak menjadi 64. Hasil ini menunjukkan bahwa model yang terlalu kompleks justru rentan mengalami overfitting karena ketidakmampuannya dalam menggeneralisasi data, yang diperparah oleh absennya nilai regularisasi L1 maupun L2 pada proses pembelajaran.

Berikut merupakan grafik loss dari data train dan validation.

Tabel 2.2.1.2 Hasil Grafik Loss Pengujian Depth dan Width

No.	hidden_layers	Grafik
Depth		
1	[32]	

2	[32, 32, 32]	Depth Test hidden=[32, 32, 32] (width tetap=32)
3	[32, 32, 32, 32, 32]	Depth Test hidden=[32, 32, 32, 32, 32] (width tetap=32)
Width		
1	[16, 16]	Width Test hidden=[16, 16] (depth tetap=2)
2	[32, 32]	Width Test hidden=[32, 32] (depth tetap=2)



Berdasarkan grafik grafik loss di atas, secara umum nilai training loss pada seluruh konfigurasi terus menurun seiring berjalannya iterasi, yang menunjukkan bahwa model mampu mempelajari pola dari data latih dengan baik. Namun, kurva validation loss memperjelas adanya indikasi overfitting pada arsitektur model yang terlalu kompleks. Hal ini terlihat sangat jelas pada pengaturan width terlebar ([64, 64]) dan depth terdalam ([32, 32, 32, 32, 32]), dengan garis validation loss berbalik mengalami tren kenaikan pada bagian akhir proses pembelajaran. Sebaliknya, model dengan arsitektur yang lebih sederhana, seperti dengan 1 layer ([32]) atau width terkecil ([16, 16]), menghasilkan kurva validation loss yang lebih stabil, sehingga membuktikan bahwa arsitektur tersebut memiliki kemampuan generalisasi yang lebih baik.

2.2.2 Pengaruh Fungsi Aktivasi

Pengujian ini dilakukan pada 7 fungsi aktivasi yang berbeda dengan variabel kontrol learning rate sebesar 0.01, hidden layer [64, 32, 16] dan tidak ada regularisasi l1 dan l2. Berikut merupakan hasilnya.

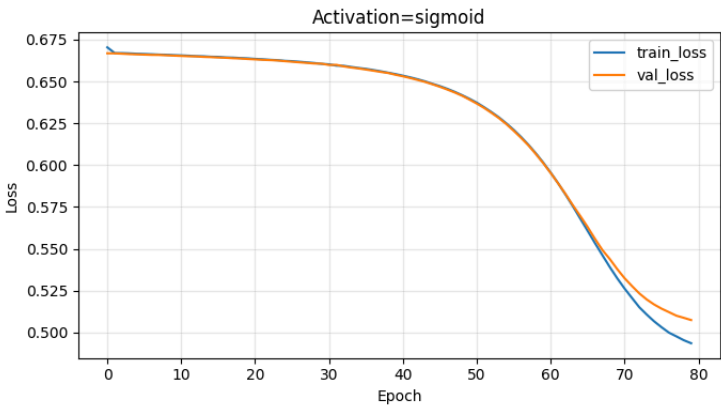
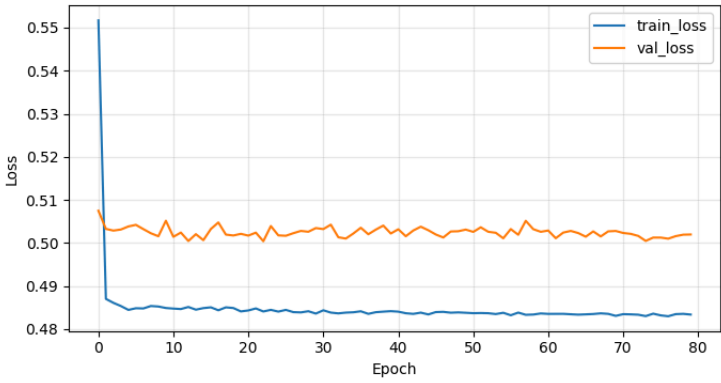
Tabel 2.2.2.1 Hasil Metrik Pengujian Fungsi Aktivasi

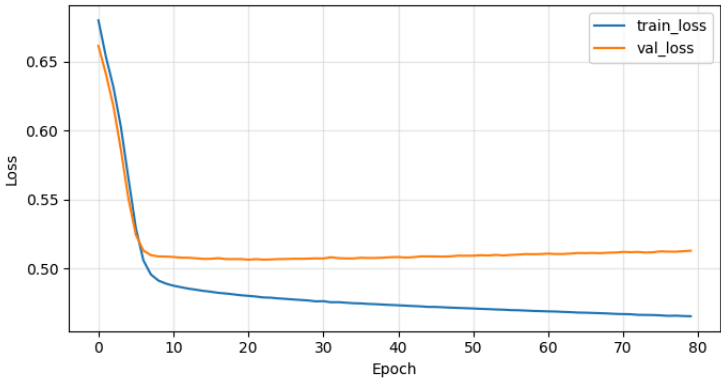
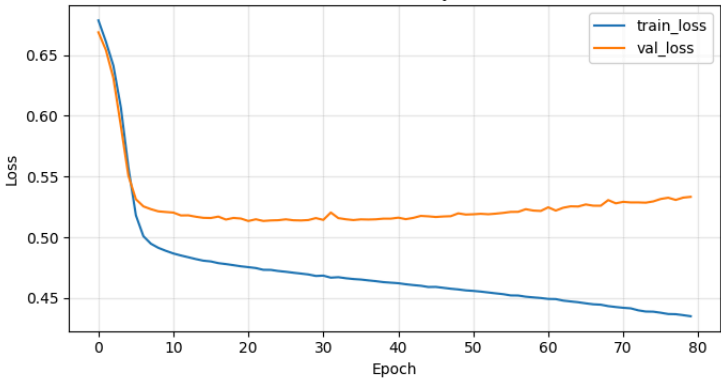
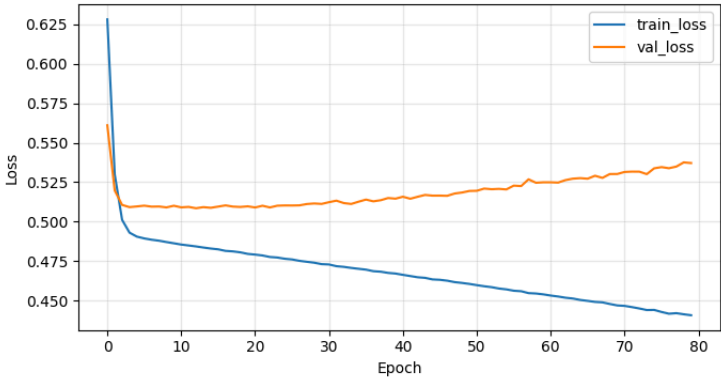
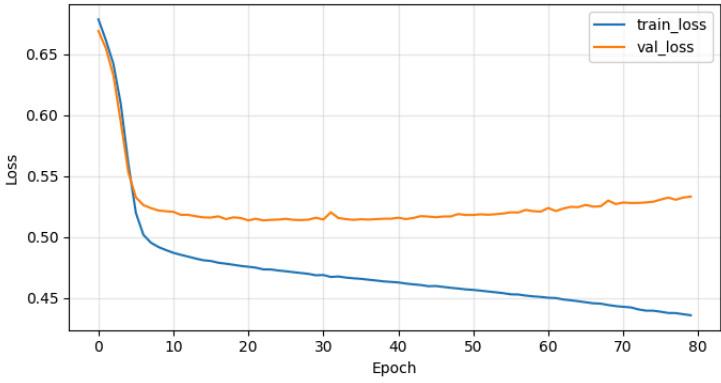
No.	Fungsi Aktivasi	Accuracy	Precision	Recall	F1
1	Sigmoid	0.751333	0.768031	0.853738	0.808620
2	Linear	0.751333	0.769608	0.850488	0.808029
3	Swish	0.736000	0.762712	0.828819	0.794393
4	LeakyReLU	0.739333	0.774227	0.813651	0.793450
5	Tanh	0.734667	0.762763	0.825569	0.792924
6	ReLU	0.736667	0.771605	0.812568	0.791557
7	Softmax	0.615333	0.615333	1.000000	0.761865

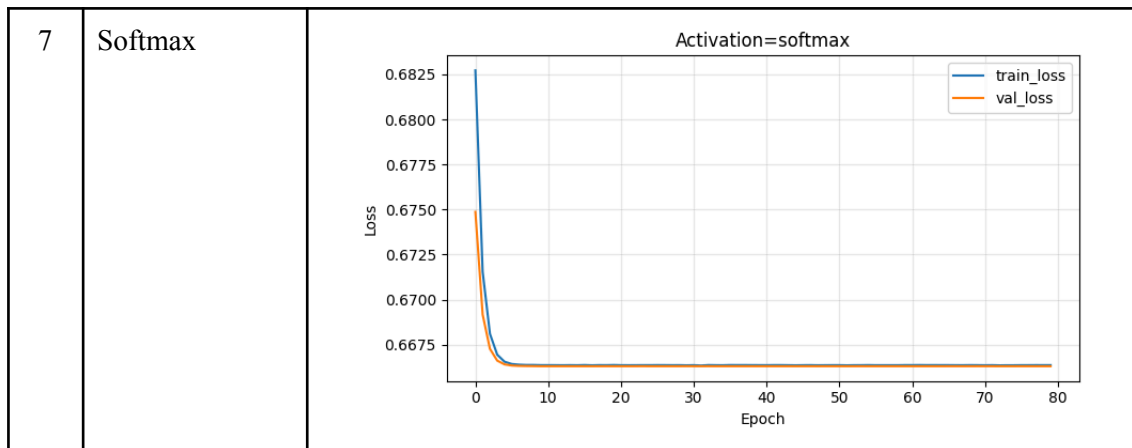
Berdasarkan hasil pengujian variasi fungsi aktivasi, fungsi Sigmoid dan Linear secara konsisten menunjukkan performa terbaik dengan tingkat akurasi tertinggi masing-masing 75,13% dan F1-score sekitar 0,808. Kelompok fungsi modern seperti Swish, LeakyReLU, dan ReLU, termasuk fungsi tradisional Tanh, memberikan hasil yang sedang dengan akurasi sekitar 73% dan performa yang relatif berdekatan. Sebaliknya, fungsi Softmax memberikan performa akurasi terburuk di antara seluruh pengujian meskipun memiliki nilai recall sempurna, yang secara umum menandakan bahwa model cenderung memprediksi suatu kelas dengan generalisasi yang tinggi dan tidak cocok digunakan pada arsitektur hidden layer. Hasil ini mengindikasikan bahwa pemetaan yang lebih linier atau probabilitas logistik sederhana lebih efektif dalam mempelajari karakteristik data latih tersebut dibandingkan pemetaan berbasis terpotong atau eksponensial.

Adapun grafik loss dari setiap fungsi aktivasi seperti berikut.

Tabel 2.2.2.2 Hasil Grafik Loss Pengujian Fungsi Aktivasi

No.	Fungsi Aktivasi	Grafik Loss
1	Sigmoid	
2	Linear	

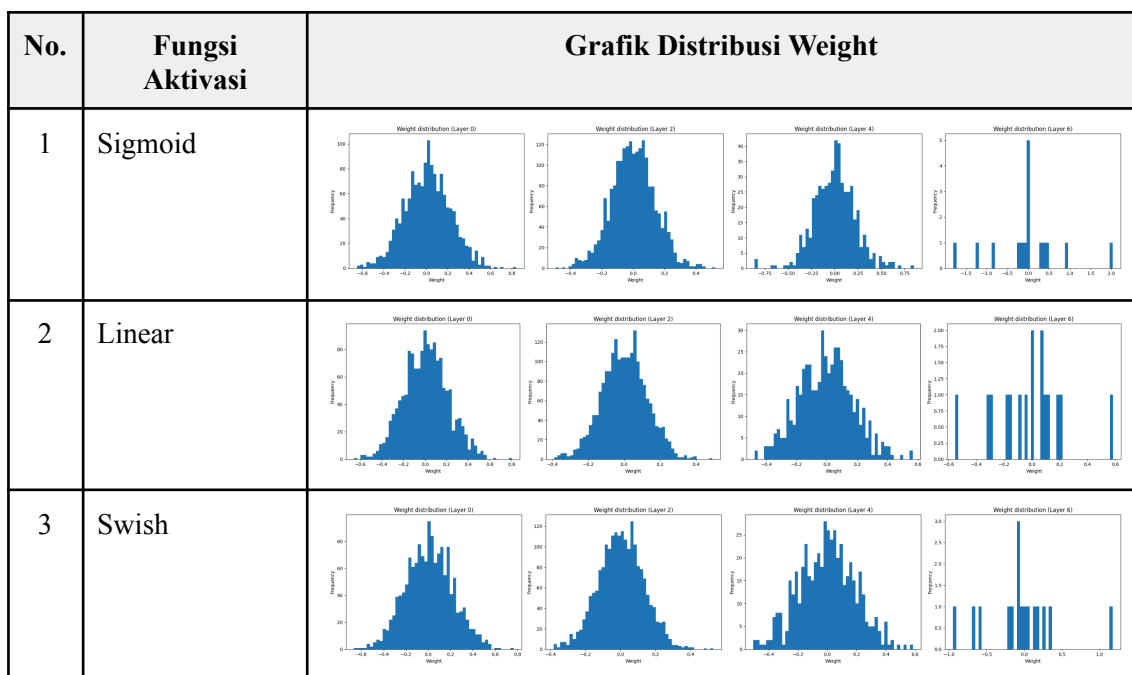
3	Swish	Activation=swish  Loss Epoch train_loss val_loss
4	LeakyReLU	Activation=leakyrelu  Loss Epoch train_loss val_loss
5	Tanh	Activation=tanh  Loss Epoch train_loss val_loss
6	ReLU	Activation=relu  Loss Epoch train_loss val_loss

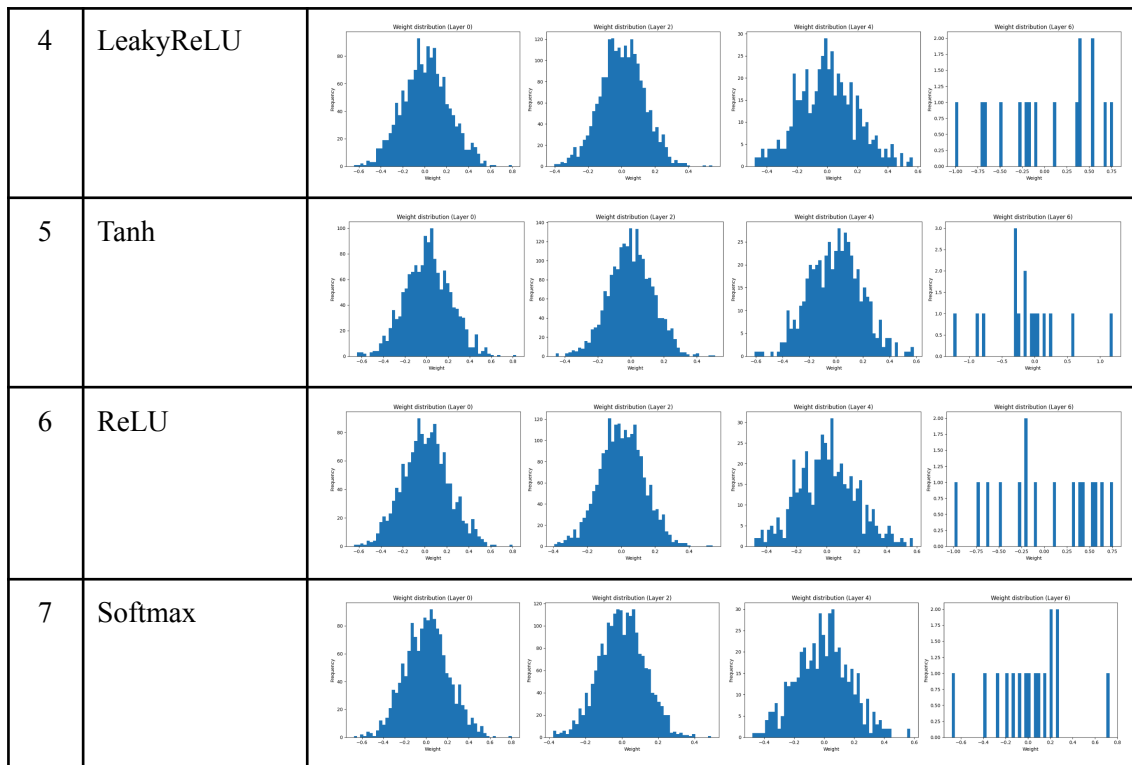


Berdasarkan grafik loss, fungsi aktivasi Sigmoid menunjukkan kurva konvergensi yang paling stabil dan mulus. Berdasarkan pengujian, fungsi Swish menunjukkan pola yang sangat mirip dengan ReLU dalam kecepatan konvergensi, namun nilai validation loss yang cenderung lebih stabil dan sedikit memburuk seiring bertambahnya epoch. Di sisi lain, LeakyReLU awalnya juga berkinerja menyerupai ReLU, tetapi nilai loss pada data validasi justru semakin memburuk seiring berjalannya epoch, sedangkan Softmax dan Linear sangat cepat dalam konvergensi namun langsung seperti terkena plateau dan tidak dapat menekan nilai loss lagi.

Berikut merupakan grafik distribusi dari weight dan gradien tiap fungsi aktivasi.

Tabel 2.2.2.3 Hasil Grafik Distribusi Weight Pengujian Fungsi Aktivasi

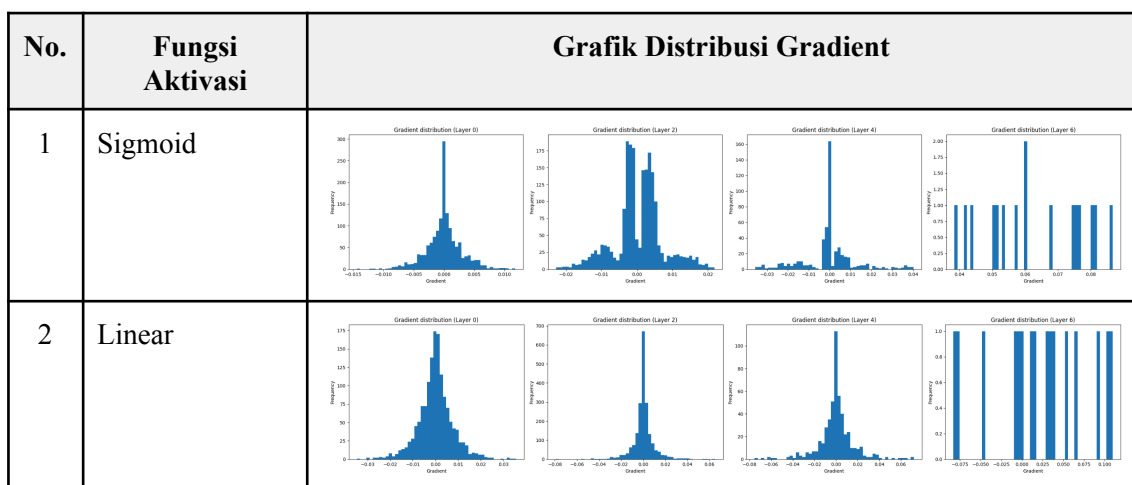


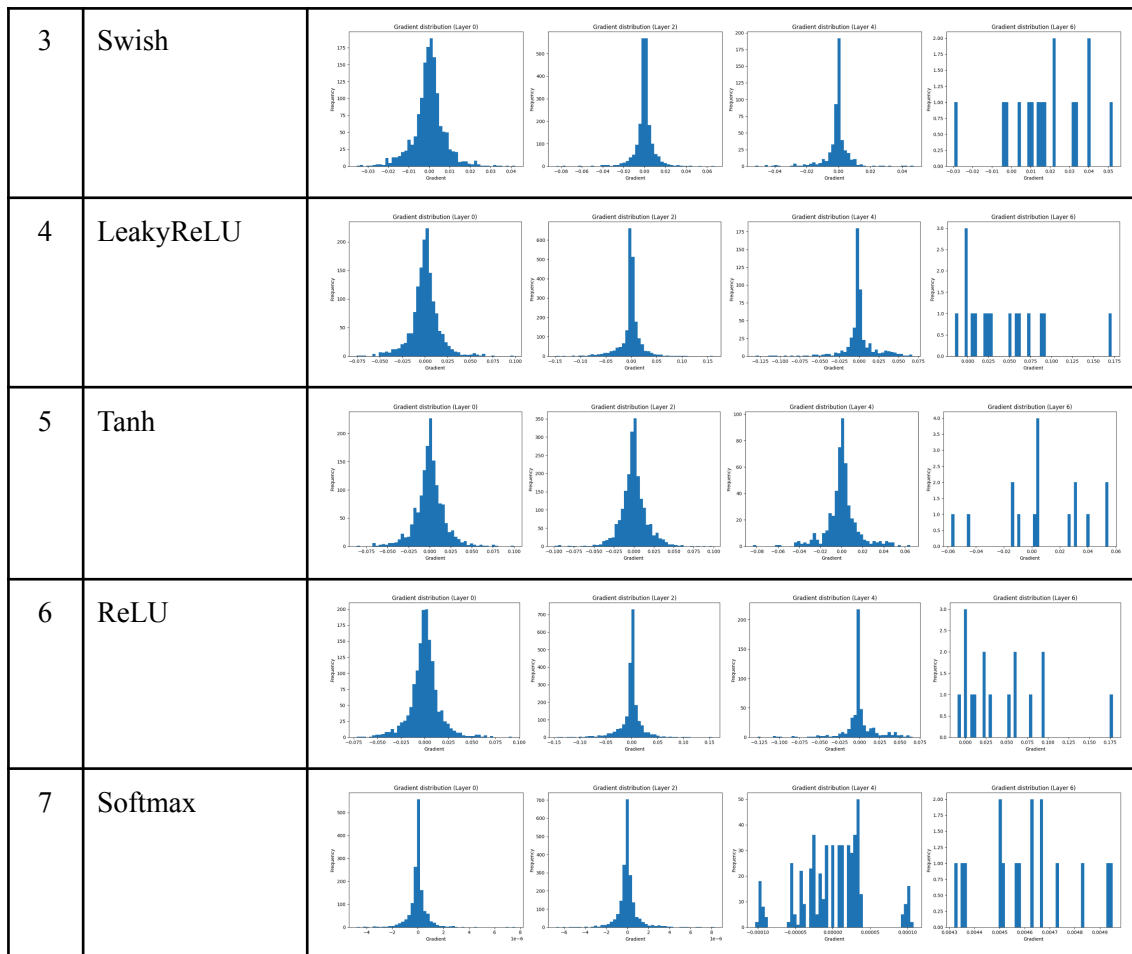


Grafik distribusi bobot menunjukkan pola konvergensi yang sangat konsisten untuk semua fungsi aktivasi yang diuji. Pada seluruh hidden layer, bobot menyebar membentuk pola kurva normal yang terpusat di angka nol dengan rentang stabil, menandakan tidak terjadinya *exploding weights* selama proses training. Bentuk grafik pada layer terakhir diakibatkan oleh sedikitnya jumlah parameter output. Secara keseluruhan, jenis fungsi aktivasi tidak memberikan perbedaan visual yang signifikan terhadap profil penyebaran bobot jaringan.

Berikut merupakan grafik distribusi gradien dari setiap fungsi aktivasi.

Tabel 2.2.2.4 Hasil Grafik Distribusi Gradien Pengujian Fungsi Aktivasi





Fungsi aktivasi seperti Sigmoid, Tanh, dan khususnya Softmax menunjukkan indikasi terjadinya *vanishing gradient*, ditandai dengan nilai gradien memusat secara ekstrem di sekitar angka nol dengan rentang yang sangat kecil, yang akan menghambat pembaruan bobot pada layer-layer awal saat proses training. Sebaliknya, kelompok fungsi aktivasi modern seperti ReLU, Leaky ReLU, dan Swish mampu mempertahankan sebaran gradien yang jauh lebih baik dan stabil di berbagai layer. Pada Leaky ReLU dan Swish, distribusi gradien tetap terjaga dengan baik dan mencegah nilai gradien mati sepenuhnya, membuktikan efektivitas dalam menjaga stabilitas proses backpropagation.

2.2.3 Pengaruh Learning Rate

Pengujian learning rate dilakukan dengan variabel kontrol hidden layer [64, 32, 16], fungsi aktivasi ReLU, dan tidak dilakukan regularisasi l1 dan l2. Berikut merupakan hasil pengujiannya.

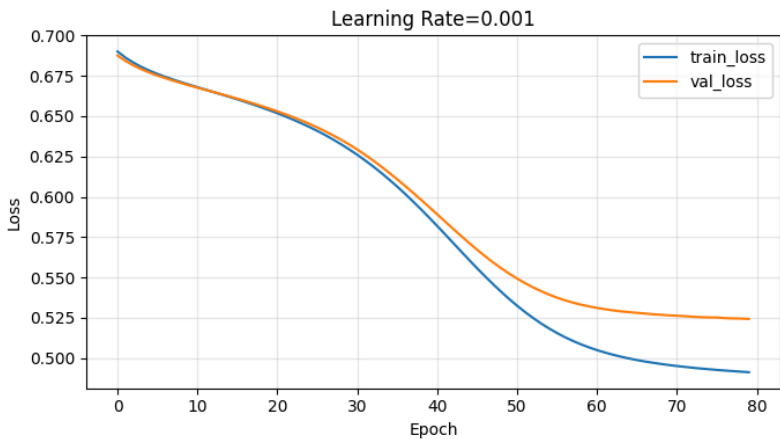
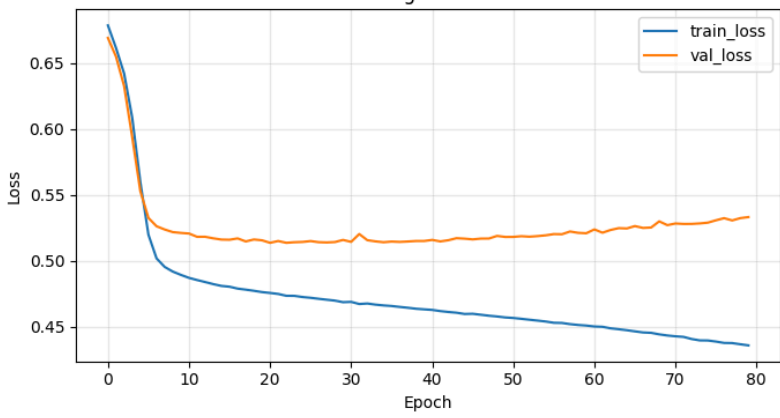
Tabel 2.2.3.1 Hasil Metrik Pengujian Nilai Learning Rate

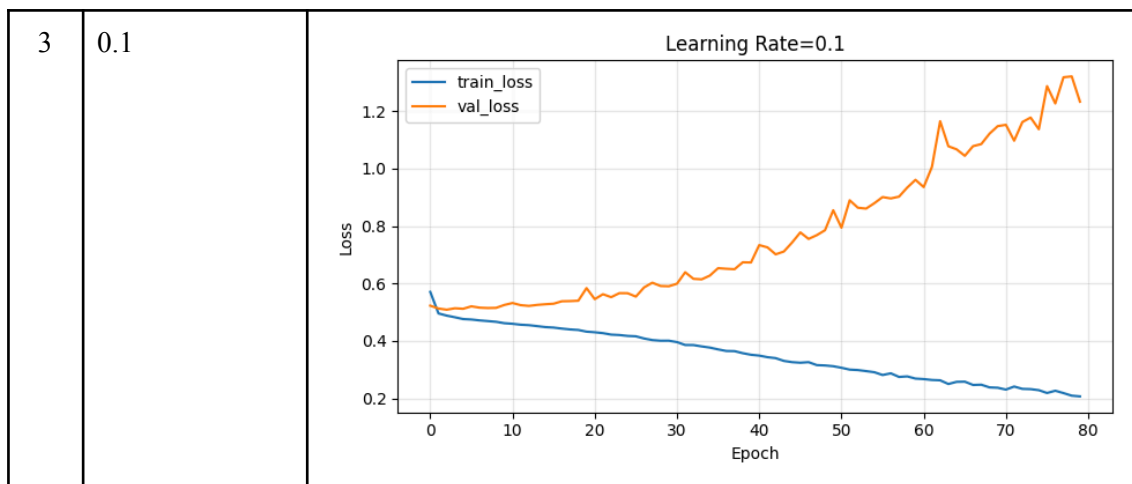
No.	Nilai Learning Rate	Accuracy	Precision	Recall	F1
1	0.001	0.728000	0.752699	0.830986	0.789907

2	0.01	0.736667	0.771605	0.812568	0.791557
3	0.1	0.670000	0.738839	0.717226	0.727872

Berdasarkan hasil pada tabel pengujian di atas, variasi learning rate memberikan pengaruh yang signifikan terhadap kinerja model. Learning rate sebesar 0,01 menghasilkan akurasi tertinggi dibandingkan dengan nilai lainnya, diikuti oleh learning rate 0,001, sementara learning rate 0,1 menghasilkan akurasi terendah. Hal ini menunjukkan bahwa learning rate yang terlalu kecil menyebabkan proses pembelajaran berlangsung lambat sehingga model belum mencapai performa optimal, sedangkan learning rate yang terlalu besar menyebabkan penurunan performa akibat pembaruan bobot yang terlalu besar. Dengan demikian, nilai learning rate yang moderat memberikan keseimbangan terbaik antara kecepatan pembelajaran dan kualitas hasil akhir (F1-Score). Kemudian, di bawah ini merupakan tabel grafik loss dari setiap fungsi aktivasi.

Tabel 2.2.3.2 Hasil Grafik Loss Pengujian Nilai Learning Rate

No.	Nilai Learning Rate	Grafik Loss
1	0.001	
2	0.01	



Dari hasil grafik loss di atas, perbedaan learning rate menghasilkan pola konvergensi yang berbeda. Pada learning rate 0.001, kurva loss untuk training dan validation menunjukkan penurunan yang stabil namun berlangsung secara lambat. Hal ini menandakan proses pembelajaran yang kurang efisien.

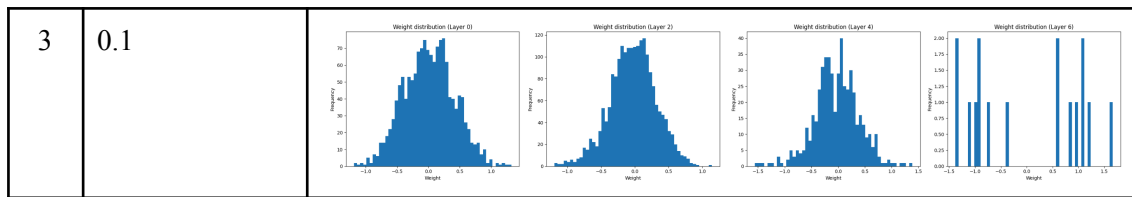
Pada learning rate 0.01, kurva loss menurun lebih cepat dan stabil menuju titik konvergensi. Untuk kurva loss training dan validation, penurunan nilai loss cukup signifikan pada epoch 0 hingga 5, kemudian mengalami perlambatan penurunan nilai loss setelah epoch lebih dari 5. Hal ini menunjukkan bahwa model mampu belajar secara efektif.

Sebaliknya, pada learning rate 0,1, kurva loss cenderung berfluktuasi dan tidak menunjukkan konvergensi yang jelas. Untuk kurva loss validation memiliki penurunan yang cukup stabil sementara untuk kurva loss training mengalami kenaikan. Hal ini mengindikasikan adanya ketidakstabilan dalam proses pelatihan akibat pembaruan parameter yang terlalu besar.

Berikut merupakan grafik distribusi dari weight dan gradien tiap fungsi aktivasi.

Tabel 2.2.3.3 Hasil Grafik Distribusi Weight Pengujian Nilai Learning Rate

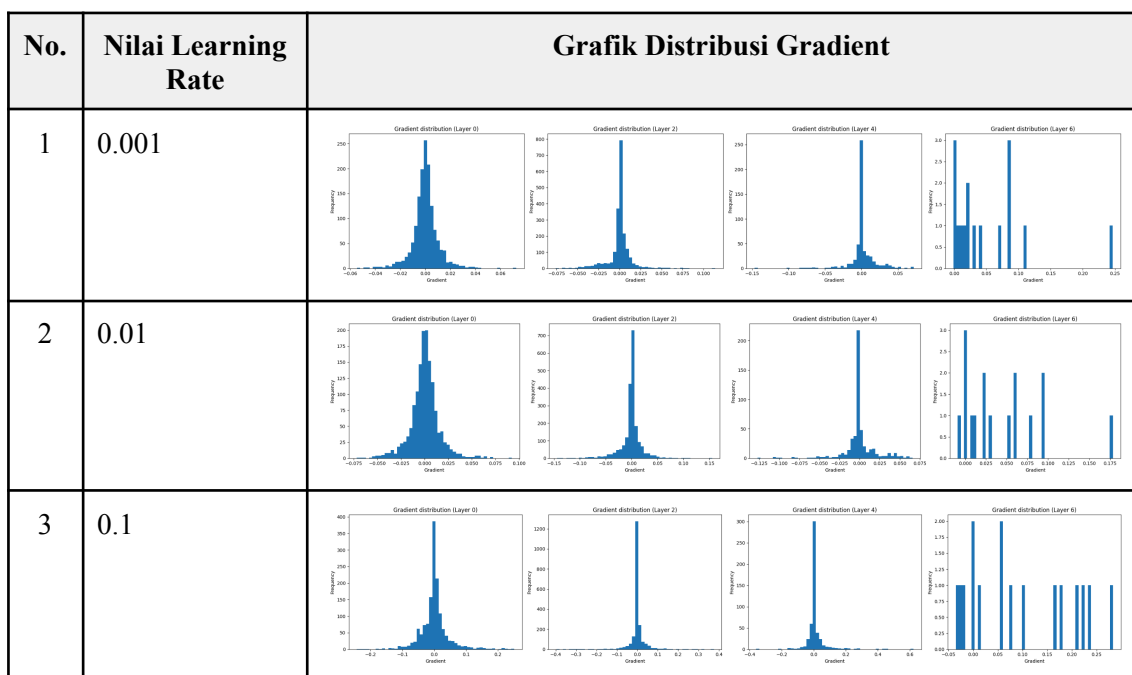
No.	Nilai Learning Rate	Grafik Distribusi Weight
1	0.001	
2	0.01	



Pada hasil distribusi bobot (*weight*) di atas, dapat diambil informasi bahwa learning rate memengaruhi seberapa jauh parameter model berubah selama pelatihan. Pada learning rate 0.001, distribusi bobot cenderung sempit yang menunjukkan bahwa perubahan bobot berlangsung secara bertahap dan terbatas. Pada learning rate 0.01, distribusi bobot menjadi lebih menyebar namun tetap terkontrol, yang menandakan bahwa model mampu mengeksplorasi ruang parameter secara efektif. Sementara itu, pada learning rate 0.1, distribusi bobot terlihat lebih lebar dan kurang stabil yang menandakan bahwa pembaruan bobot terlalu besar dan tidak terkontrol dengan baik.

Berikut merupakan grafik distribusi gradien dari setiap fungsi aktivasi.

Tabel 2.2.3.4 Hasil Grafik Distribusi Gradien Pengujian Nilai Learning Rate



Dari hasil distribusi gradien, learning rate juga memiliki pengaruh terhadap stabilitas pembelajaran. Pada learning rate 0.001, gradien cenderung kecil dan terkonsentrasi di sekitar nol yang menyebabkan pembaruan parameter menjadi sangat terbatas. Pada learning rate 0.01, distribusi gradien lebih bervariasi namun masih dalam rentang yang stabil sehingga mendukung proses pembelajaran yang efektif. Sebaliknya, pada learning rate 0.1, distribusi gradien menunjukkan nilai yang lebih besar yang membuktikan adanya ketidakstabilan dalam proses optimisasi dan berpotensi menyebabkan model gagal mencapai konvergensi.

2.2.4 Pengaruh Inisialisasi Bobot

Pengujian inisialisasi bobot dilakukan dengan variabel kontrol epoch 80, batch size 32, learning rate 0.01, fungsi aktivasi ReLU, hidden layer [64, 32, 16], dan tidak ada regularisasi l1 dan l2. Berikut merupakan hasil pengujiannya.

Tabel 2.2.4.1 Hasil Metrik Pengujian Metode Inisialisasi Bobot

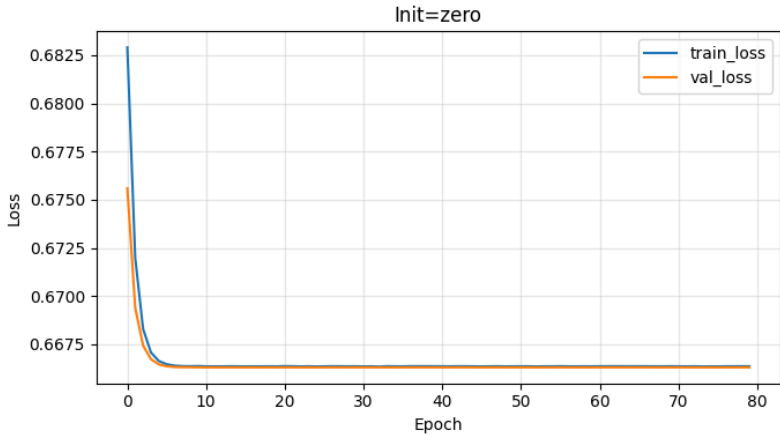
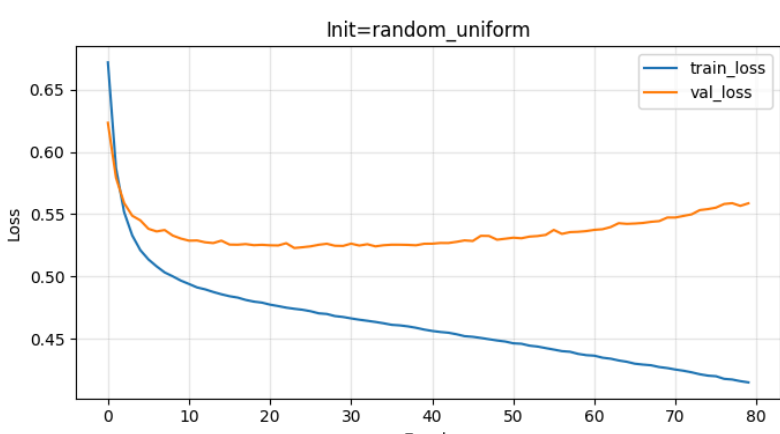
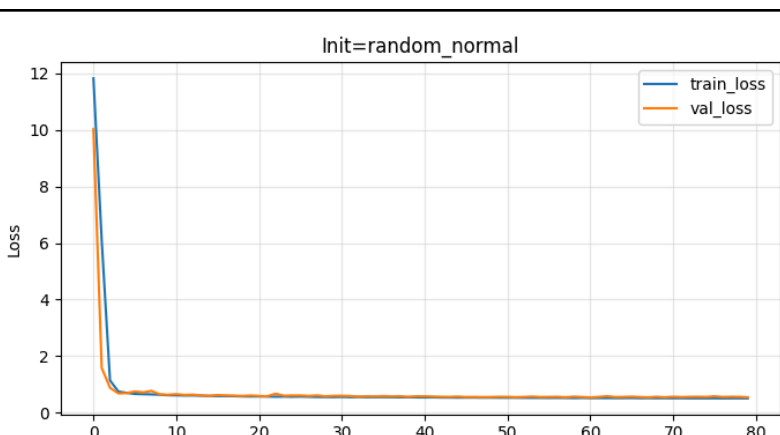
No.	Metode Inisialisasi Bobot	Accuracy	Precision	Recall	F1
1	Zero	0.615333	0.615333	1.000000	0.761865
2	Random Uniform	0.733333	0.775553	0.797400	0.786325
3	Random Normal	0.721333	0.718994	0.898158	0.79865
4	He	0.727333	0.767152	0.799567	0.783024
5	Xavier	0.736667	0.771605	0.812568	0.791557

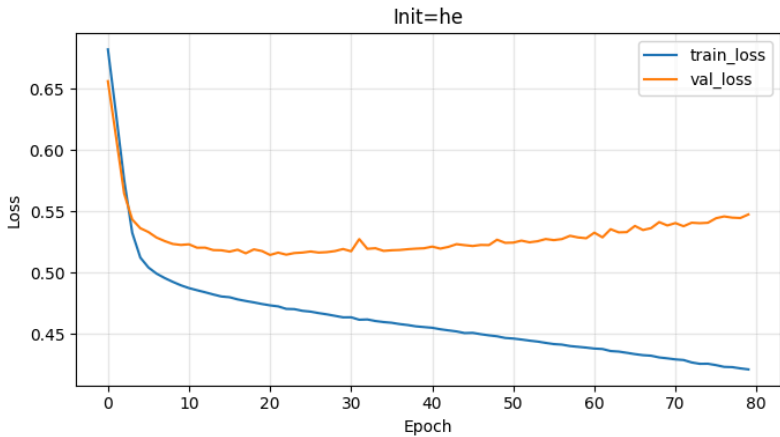
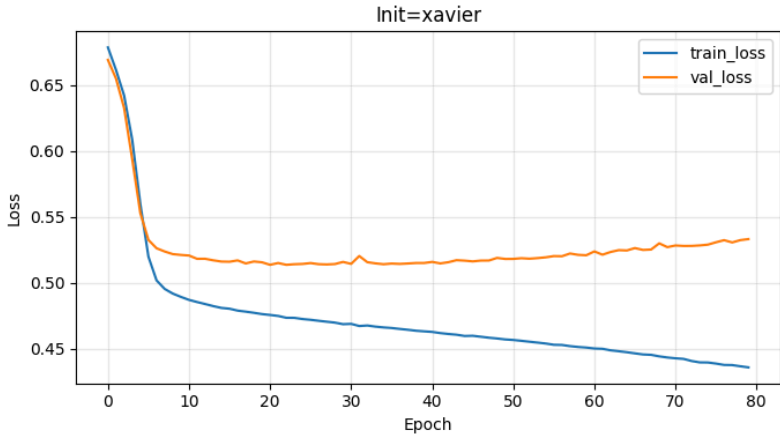
Pada hasil metrik pengujian, dapat diambil informasi bahwa dua metode yang memiliki F1-Score terbaik adalah metode Random Normal dan Xavier. F1-Score tertinggi dimiliki oleh Random Normal (0.798650) kemudian diikuti oleh Xavier (0.791557). Meskipun begitu, nilai precision dan untuk metode Xavier (0.771605) lebih tinggi dari metode Random Normal (0.718994). Dengan begitu, metode Random Normal memiliki keberhasilan prediksi yang lebih akurat namun metode Xavier lebih baik dalam memprediksi kelas positif. Untuk metode inisialisasi bobot lainnya seperti Zero, Random Uniform, dan He masih belum cukup stabil untuk performa.

Adapun grafik loss dari setiap metode inisialisasi bobot seperti berikut.

Tabel 2.2.4.2 Hasil Grafik Loss Pengujian Metode Inisialisasi Bobot

No.	Metode Inisialisasi Bobot	Grafik Loss
-----	---------------------------	-------------

1	Zero	 Init=zero Y-axis: Loss (0.6675 to 0.6825) X-axis: Epoch (0 to 80) Legend: train_loss (blue), val_loss (orange)
2	Random Uniform	 Init=random_uniform Y-axis: Loss (0.45 to 0.65) X-axis: Epoch (0 to 80) Legend: train_loss (blue), val_loss (orange)
3	Random Normal	 Init=random_normal Y-axis: Loss (0 to 12) X-axis: Epoch (0 to 80) Legend: train_loss (blue), val_loss (orange)

4	He	
5	Xavier	

Jika dikaitkan dengan grafik loss, perbedaan performa antar metode inisialisasi bobot dapat dijelaskan melalui pola konvergensinya. Pada inisialisasi zero, kurva loss mengalami penurunan yang signifikan pada epoch kecil tetapi berhenti di nilai sekitar 0.665 di epoch-epoch berikutnya. Hal ini mengindikasikan bahwa model tidak benar-benar belajar pola, melainkan hanya menghasilkan prediksi yang bias.

Pada metode random uniform, He, dan Xavier, untuk kurva loss pada validation sudah mulai menunjukkan adanya penurunan yang stabil pada epoch yang besar tetapi kurva loss validation malah cenderung naik di epoch yang besar.

Kemudian pada metode random normal, sama dengan metode Zero, kurva loss mengalami penurunan yang signifikan di epoch kecil namun yang membedakan dengan metode Zero adalah kurva loss tidak stagnan di nilai yang besar. Metode random normal memberi nilai loss yang hampir sama dengan metode random uniform, He, dan Xavier pada epoch yang besar.

Berikut merupakan grafik distribusi dari weight dan gradien tiap metode inisialisasi bobot.

Tabel 2.2.4.3 Hasil Grafik Distribusi Weight Pengujian Metode Inisialisasi Bobot

No.	Metode Inisialisasi Bobot	Grafik Distribusi Weight
1	Zero	
2	Random Uniform	
3	Random Normal	
4	He	
5	Xavier	

Berdasarkan hasil di atas, pada inisialisasi zero, seluruh bobot memiliki nilai yang sama sehingga tidak terdapat keberagaman antar neuron, yang menyebabkan model gagal membedakan fitur secara efektif. Pada metode random uniform dan random normal, distribusi bobot mulai menyebar, namun karakteristik distribusinya berbeda. Random normal cenderung menghasilkan nilai yang lebih terkonsentrasi di sekitar rata-rata dengan beberapa nilai ekstrem, yang dapat memengaruhi stabilitas pembelajaran. Sementara itu, metode He dan Xavier menghasilkan distribusi bobot yang lebih seimbang dan terkontrol, dengan penyebaran yang proporsional terhadap ukuran layer. Kondisi ini memungkinkan setiap neuron memiliki peran yang berbeda sejak awal, sehingga meningkatkan kemampuan model dalam menangkap pola data.

Berikut merupakan grafik distribusi gradien dari setiap metode inisialisasi bobot.

Tabel 2.2.4.4 Hasil Grafik Distribusi Gradien Pengujian Metode Inisialisasi Bobot

No.	Metode Inisialisasi Bobot	Grafik Distribusi Gradien
1	Zero	
2	Random Uniform	
3	Random Normal	
4	He	
5	Xavier	

Analisis distribusi gradien menunjukkan perbedaan yang konsisten dengan hasil metrik. Pada inisialisasi zero, gradien cenderung sangat kecil dan tidak bervariasi, yang menyebabkan pembaruan bobot menjadi tidak efektif dan menghambat proses pembelajaran. Pada metode random uniform dan He, distribusi gradien mulai menunjukkan variasi yang lebih luas, namun pada beberapa kasus dapat menjadi tidak seimbang, yang tercermin dari perbedaan antara precision dan recall. Sementara itu, metode random normal dan Xavier menghasilkan distribusi gradien yang lebih stabil dan seimbang. Untuk metode Xavier, nilai distribusi gradien tidak terlalu kecil maupun terlalu besar namun untuk metode random normal tersebar di nilai yang lebih kecil dari metode Xavier. Stabilitas gradien ini berkontribusi langsung terhadap keseimbangan performa metrik.

2.2.5 Pengaruh Regularisasi

Pada pengujian ini, dilakukan evaluasi terhadap penggunaan teknik regularisasi pada model FFNN. Pengujian menggunakan *base architecture* dengan *hidden layer* [64, 32, 16], fungsi

aktivasi ReLU, dan *learning rate* 0.01. Model dievaluasi dalam tiga kondisi, yaitu tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2. Berikut tabel hasil akhir prediksi:

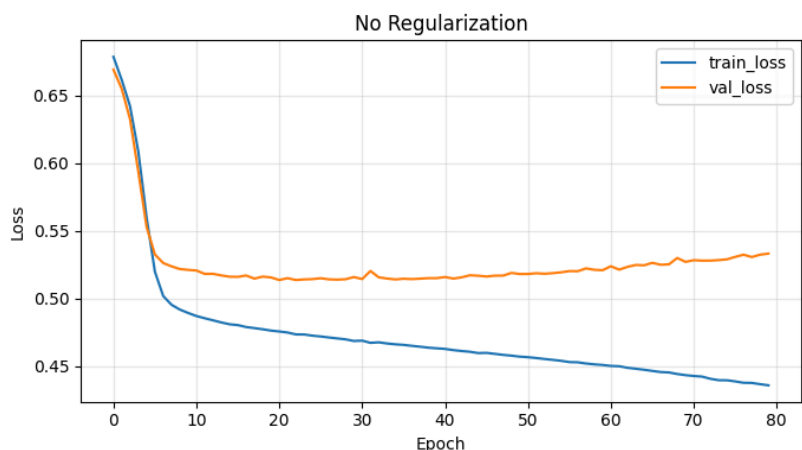
Tabel 2.2.5.1 Hasil Metrik Weight Pengujian Regularisasi

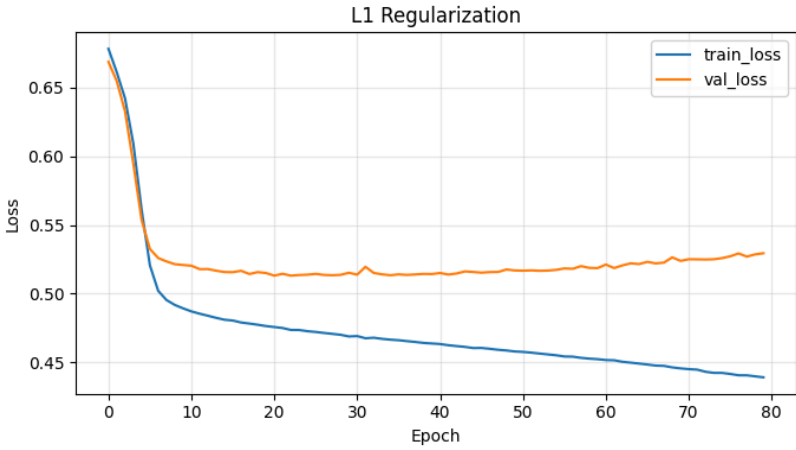
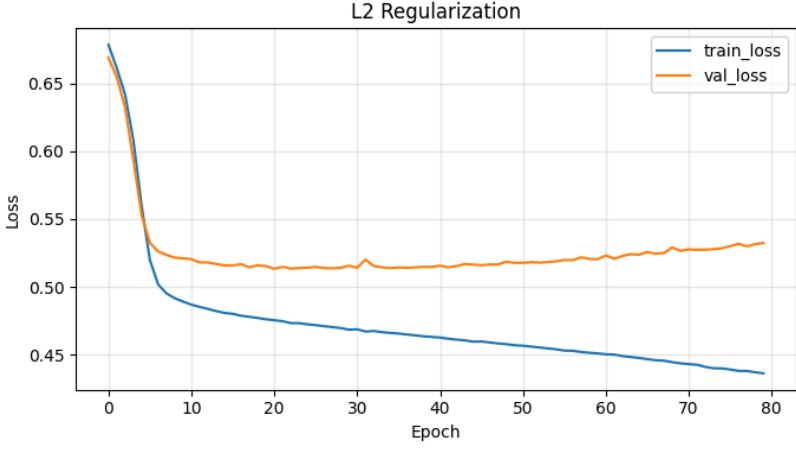
Model	Accuracy	Precision	Recall	F1 Score
No Regularization	0.73666	0.771605	0.812568	0.791557
L1 Regularization	0.736667	0.772165	0.811484	0.791337
L2 Regularization	0.734667	0.770340	0.810401	0.789863

Berdasarkan hasil, ketiga model menunjukkan performa yang cenderung mirip, dengan selisih sedikit. Model tanpa regularisasi (*No Regularization*) justru menghasilkan nilai F1-Score (0.7915) dan Recall (0.8125) yang paling tinggi. Penambahan regularisasi L1 memberikan sedikit peningkatan pada Precision (0.7721), namun menurunkan F1-Score secara keseluruhan. Di sisi lain, regularisasi L2 memberikan performa paling buruk di seluruh metrik penilaian. Hasil ini memberikan informasi bahwa arsitektur dasar model kemungkinan tidak mengalami *overfitting* yang parah pada data *training*, sehingga penambahan penalti bobot (L1 maupun L2) justru memperburuk kapasitas model dalam mempelajari pola data yang kompleks.

Selain itu, terdapat grafik loss dari setiap metode regularization dapat dilihat seperti berikut.

Tabel 2.2.5.2 Hasil Grafik Loss Pengujian Regularisasi

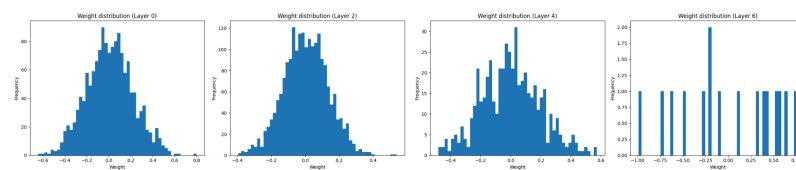
No.	Metode Regularisasi	Grafik Loss
1	No Regularization	

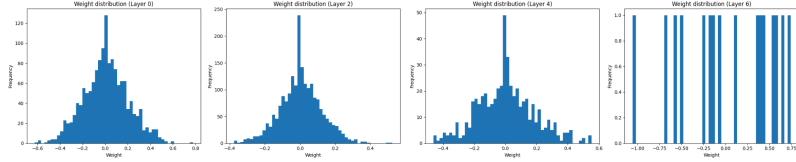
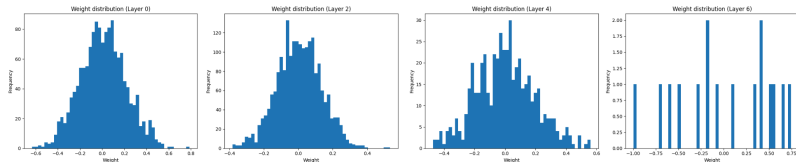
2	L1 Regularization	 The graph titled 'L1 Regularization' shows 'train_loss' (blue line) and 'val_loss' (orange line) over 80 epochs. The y-axis is 'Loss' ranging from 0.45 to 0.65. The x-axis is 'Epoch' ranging from 0 to 80. Both curves start at approximately 0.65 at epoch 0. The training loss decreases steadily to about 0.44 by epoch 80. The validation loss decreases to about 0.52 by epoch 10 and then remains relatively stable with minor fluctuations around 0.52 for the remainder of the training.
3	L2 Regularization	 The graph titled 'L2 Regularization' shows 'train_loss' (blue line) and 'val_loss' (orange line) over 80 epochs. The y-axis is 'Loss' ranging from 0.45 to 0.65. The x-axis is 'Epoch' ranging from 0 to 80. Both curves start at approximately 0.65 at epoch 0. The training loss decreases steadily to about 0.44 by epoch 80. The validation loss decreases to about 0.52 by epoch 10 and then remains relatively stable with minor fluctuations around 0.52 for the remainder of the training.

Berdasarkan perbandingan grafik *loss* di tiap epoch, dapat dilihat bahwa ketiga model menunjukkan lintasan kurva yang hampir identik tanpa ada perbedaan visual yang signifikan. Seluruh grafik menunjukkan penurunan *loss* yang tajam dan mulus pada 20 *epoch* pertama dan mulai konvergen secara stabil hingga akhir *training*. Ini menandakan penambahan regularisasi tidak memberikan *weight penalty* yang cukup dominan untuk mengubah lintasan *loss* secara kasat mata.

Terdapat juga grafik distribusi dari weight dan gradien tiap metode regularisasi yang dapat dilihat sebagai berikut.

Tabel 2.2.5.3 Hasil Grafik Distribusi Weight Pengujian Regularisasi

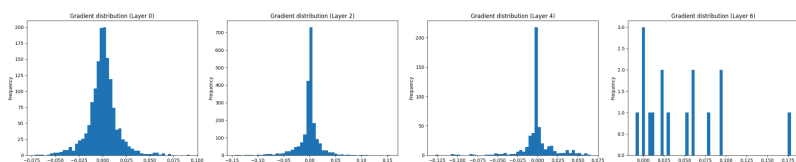
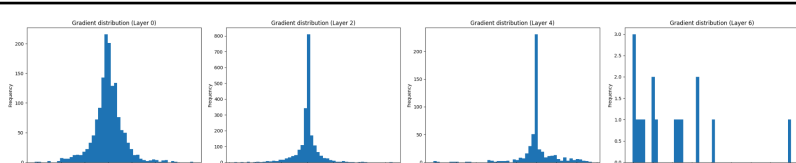
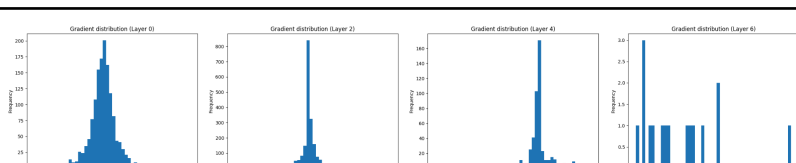
No.	Metode Regularisasi	Grafik Distribusi Weight
1	No Regularization	 Four histograms showing weight distributions for layers 0, 2, 4, and 6 without regularization. Layer 0 and 2 show normal distributions centered around 0. Layer 4 shows a distribution skewed towards positive values. Layer 6 shows a uniform distribution across the range from -1.00 to 1.00.

2	L1 Regularization	
3	L2 Regularization	

Berdasarkan visualisasi histogram distribusi bobot, model No Regularization relatif simetris di sekitar nol, menandakan training berjalan sehat, bobot tidak condong ke satu nilai. Begitu pula dengan model L2 Regularization, distribusi bobotnya tidak terlalu terkompres di nol. Sedangkan model L1 Regularization terlihat mengalami penyusutan bobot berlebih yang artinya *underfitting*.

Terakhir, terdapat grafik perbandingan distribusi gradien dari setiap metode regularisasi yang dapat dilihat pada tabel berikut:

Tabel 2.2.5.4 Hasil Grafik Distribusi Gradient Pengujian Regularisasi

No.	Metode Regularisasi	Grafik Distribusi Gradient
1	No Regularization	
2	L1 Regularization	
3	L2 Regularization	

Berdasarkan hasil grafik, pada ketiga model, distribusi gradien di layer awal dan menengah tetap terpusat di sekitar nol, sehingga proses training terlihat tidak stabil. Ketiga grafik menunjukkan *training* yang sehat, dengan perbedaan utama di tingkat konsentrasi gradien di sekitar nol. Model No Regularization paling longgar, L1 paling *sparse*, dan L2 berada di tengah.

2.2.6 Pengaruh Normalisasi RMSNorm

Pengujian ini dilakukan dengan variabel kontrol hidden layer [32, 32, 32, 32], fungsi aktivasi ReLU, learning rate 0.005, epoch 40, batch_size 32, dan tanpa regularisasi L1 dan L2. Terdapat dua model, pertama tidak memakai RMSNorm, sedangkan yang kedua memakai RMSNorm setelah setiap weight layer. Berikut merupakan hasil pengujiannya.

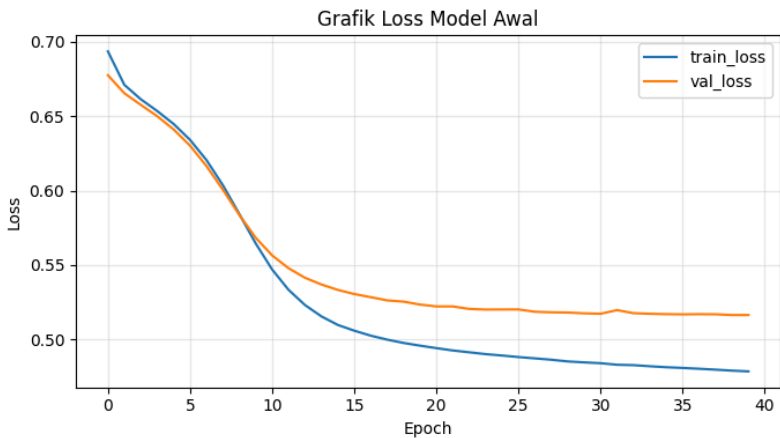
Tabel 2.2.6.1 Hasil Metrik Pengujian Normalisasi

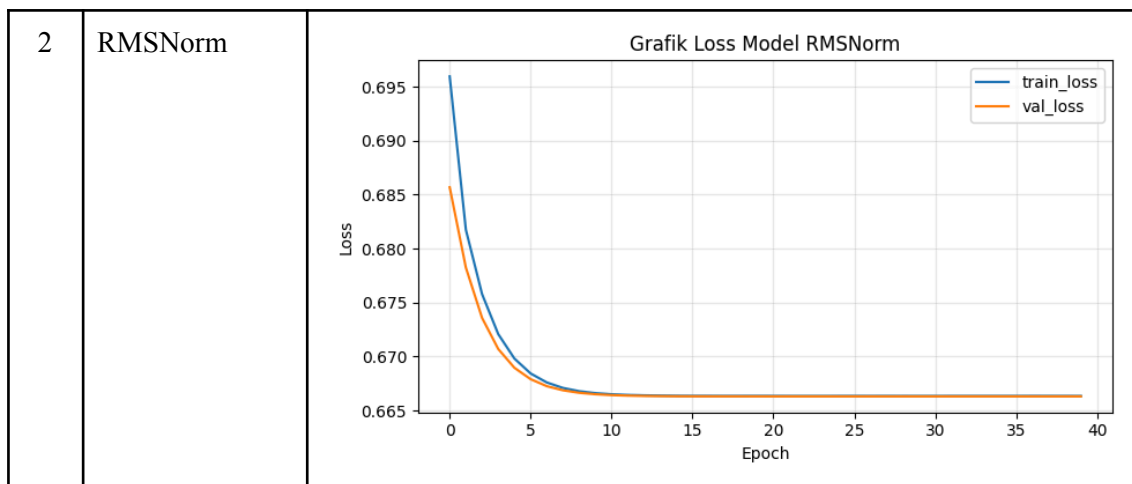
No.	Metode Normalisasi	Accuracy	Precision	Recall	F1
1	No Normalization	0.746	0.7765	0.8244	0.7997
2	RMSNorm	0.6153	0.6153	1	0.7618

Berdasarkan hasil metrik pengujian, penerapan RMSNorm pada konfigurasi tidak meningkatkan performa secara keseluruhan. Dibandingkan model tanpa normalisasi, nilai *accuracy* turun dari 0.746 menjadi 0.6153, dan F1 turun dari 0.7997 menjadi 0.7618. Hal yang membaik adalah *recall* menjadi 1.00. Ini menunjukkan model dengan RMSNorm menjadi sangat agresif untuk memprediksi kelas positif, sehingga jumlah prediksi positif yang salah meningkat.

Adapun grafik loss perbandingan penggunaan RMSNorm seperti berikut.

Tabel 2.2.6.2 Hasil Metrik Grafik Loss Pengujian Normalisasi

No.	Metode Normalisasi	Grafik Loss
1	No Normalization	



Berdasarkan hasil grafik, pada grafik model normal, *loss train* dan *validation* sama-sama turun cukup konsisten dari awal hingga akhir *epoch*. *Train loss* turun lebih jauh dibandingkan *validation loss* dan setelah sekitar *epoch 10* mulai terlihat jarak yang lumayan jelas. Pada grafik model dengan RMSNorm, *train loss* dan *validation loss* turun sangat cepat di *epoch* awal, kemudian hampir datar di nilai yang relatif tinggi. Dapat disimpulkan, RMSNorm meningkatkan stabilitas kurva *loss* pada fase awal, namun menyebabkan konvergensi awal pada nilai *loss* yang lebih tinggi.

Selain itu, terdapat juga grafik distribusi dari weight dan gradien dari perbandingan penggunaan RMSNorm yang dapat dilihat pada tabel berikut.

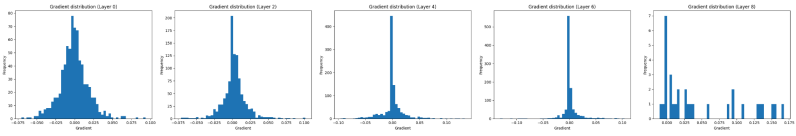
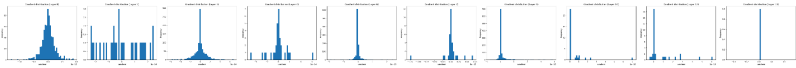
Tabel 2.2.6.3 Hasil Grafik Distribusi Weight Pengujian Normalisasi

No.	Metode Normalisasi	Grafik Distribusi Weight
1	No Normalization	
2	RMSNorm	

Berdasarkan grafik distribusi bobot, model tanpa RMSNorm cenderung lebih sehat, karena di layer-layer utama histogram terpusat di sekitar nol. Pada model dengan RMSNorm, pola bobot jadi lebih tidak seragam antara layer, di mana beberapa layer tetap berbentuk lonceng, tetapi ada layer yang sangat terkonsentrasi pada titik tertentu, mengindikasikan RMSNorm cenderung menahan perubahan parameter di layer-layer tertentu.

Terakhir, terdapat grafik distribusi gradien dari perbandingan penggunaan RMSNorm yang dapat dilihat sebagai berikut:

Tabel 2.2.6.4 Hasil Grafik Distribusi Gradien Pengujian Normalisasi

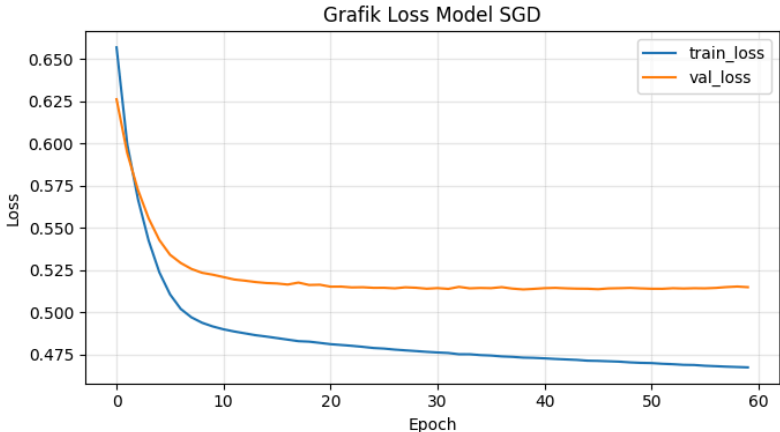
No.	Metode Normalisasi	Grafik Distribusi Gradien
1	No Normalization	
2	RMSNorm	

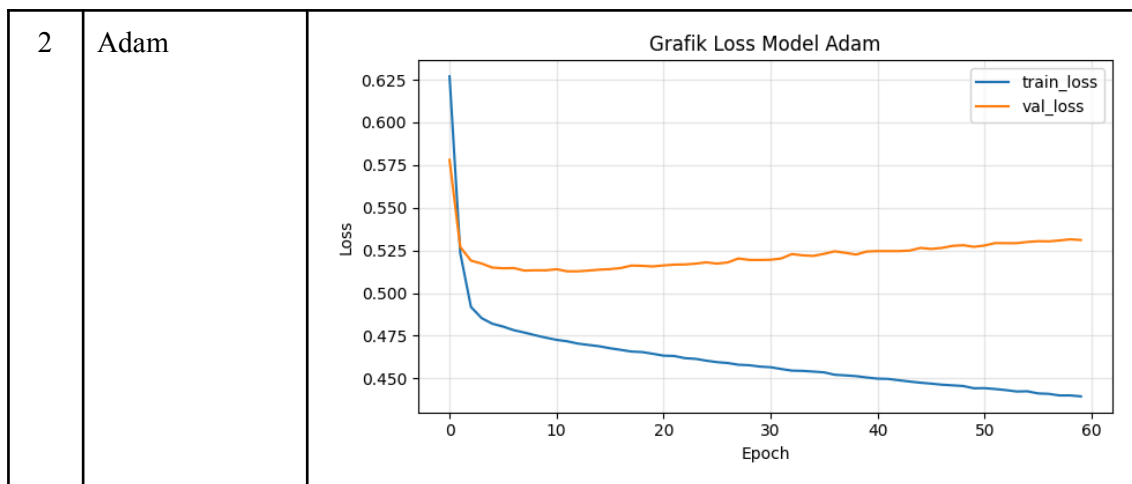
Berdasarkan hasil distribusi gradien, dapat dilihat bahwa model tanpa RMSNorm menunjukkan mayoritas gradien berada di sekitar nol dengan sebaran kiri kanan yang masih cukup lebar. Di sisi lain, model dengan RMSNorm terlihat lebih banyak spike tajam di nol dan pada beberapa layer rentang gradien sangat kecil, bahkan hampir nol. Ini berarti banyak parameter menerima update yang sangat sedikit, terutama di sebagian layer menengah atau akhir.

2.2.7 Perbandingan Metode Optimizer SGD dan Adam

Pengujian ini dilakukan untuk membandingkan kecepatan konvergensi antara dua metode yang berbeda. Variabel kontrol yang dipakai yaitu hidden layer [32, 16], epochs 60, fungsi aktivasi ReLU, batch size 32, dan tidak diberlakukan regularisasi L1 dan L2. Berikut merupakan hasil pengujiannya.

Tabel 2.2.7.1 Hasil Grafik Loss Pengujian Metode Optimizer

No.	Metode Optimizer	Grafik Loss
1	SGD	



Berdasarkan hasil pengujian, kedua *optimizer* sama-sama menunjukkan penurunan *loss* yang sangat cepat pada *epoch* awal, sehingga dapat dikatakan keduanya dapat menangkap pola dasar sejak fase awal *training*. Perbedaan terlihat setelah fase awal, di mana SGD memiliki kurva *validation loss* turun lalu stabil, sementara model Adam turun di awal tetapi kemudian naik perlahan seiring bertambahnya *epoch*. Dapat dilihat bahwa Adam unggul dalam kecepatan konvergensi data latih, sedangkan SGD lebih baik dari sisi kestabilan generalisasi data validasi.

2.2.8 Perbandingan dengan Library Sklearn

Pengujian ini dilakukan untuk membandingkan performa model Feedforward Neural Network yang dibuat dari awal (*scratch*) dengan model MLPClassifier dari scikit-learn menggunakan konfigurasi terbaik hasil eksperimen sebelumnya. Parameter yang digunakan pada MLPClassifier disesuaikan supaya setara dengan model FFNN yang dibangun dari awal. Hasil prediksi kedua model kemudian dibandingkan menggunakan metrik, di antaranya: accuracy, precision, recall, dan F1-score. Berikut di bawah merupakan tabel hasil perbandingan FFNN dari *scratch* dengan model MLPClassifier.

Tabel 2.2.8.1 Hasil Metrik Pengujian Perbandingan dengan Library Sklearn

Model	Accuracy	Precision	Recall	F1-Score
FFNN Scratch	0.758667	0.788284	0.830986	0.809072
Sklearn MLPClassifier	0.615333	0.615333	1.000000	0.761865

Hasil perbandingan menunjukkan bahwa model FFNN yang dibangun memiliki performa lebih baik dibandingkan MLPClassifier dari scikit-learn pada dataset ini. FFNN Scratch menghasilkan nilai accuracy, precision, dan F1-score yang lebih tinggi. Hal ini menunjukkan bahwa model FFNN melakukan prediksi yang seimbang antara kelas positif dan negatif. Sementara itu, model MLPClassifier dari scikit-learn memiliki recall yang sangat tinggi yaitu bernilai 1.0. Hal ini menunjukkan bahwa hampir semua data positif berhasil terdeteksi namun

precision dan accuracy-nya lebih rendah sehingga model cenderung memprediksi sebagian besar data sebagai positif (memiliki banyak nilai false positive). Hal ini menyebabkan F1-score model FFNN tetap lebih baik karena mampu menjaga keseimbangan antara nilai precision dan recall. Perbedaan performa ini dapat disebabkan oleh penyesuaian parameter, regularisasi, atau implementasi training yang lebih optimal pada model FFNN. Secara keseluruhan, model FFNN terbukti lebih efektif dalam mengenali kedua kelas secara proporsional dan memberikan hasil prediksi yang lebih baik pada pengujian ini.

3 Kesimpulan dan Saran

3.1 Kesimpulan

Berdasarkan seluruh rangkaian eksperimen dan pengujian yang telah dilakukan di tugas besar ini terhadap model *Feedforward Neural Network* (FFNN) yang dibangun *from scratch*, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Pengaruh Arsitektur (*Depth and Width*)

Penambahan jumlah *hidden layer* (*depth*) dan *neuron* (*width*) secara berlebihan tidak berbanding lurus dengan peningkatan performa model. Arsitektur yang semakin kompleks justru dapat memicu ketidakstabilan proses *training* dan *overfitting*, sementara arsitektur yang terlalu simpel seringkali gagal menangkap pola *non-linear* pada dataset.

2. Peran Fungsi Aktivasi

Penggunaan fungsi aktivasi ReLU pada *hidden layer* terbukti memberikan stabilitas konvergensi yang paling baik dibandingkan fungsi Sigmoid atau Tanh. Kedua fungsi ini dapat memicu masalah *vanishing gradient* pada arsitektur yang cukup dalam, yang ditandai dengan banyak distribusi gradien di angka nol.

3. Sensitivitas Hyperparameter

Proses pembelajaran dinilai sangat sensitif terhadap nilai *learning rate*. Nilai yang terlalu besar menyebabkan model gagal menemukan *global minimum* (divergen), sedangkan nilai yang terlalu kecil membuat *training loss* turun dengan sangat lambat.

4. Efektivitas Regularisasi

Penambahan penalti bobot seperti regularisasi L1 dan L2 sangat bergantung pada kondisi base model dan nilai λ yang digunakan. Pada eksperimen ini, menggunakan nilai 10^{-4} terbukti terlalu kecil untuk memberikan efek sparsity atau weight decay yang terlihat, sehingga model tanpa regularisasi justru lebih unggul karena pergerakan bobotnya tidak dibatasi.

5. Perbandingan dengan Scikit-Learn

Model FFNN yang dibangun *from scratch* mampu memberikan performa prediksi (*accuracy* dan F1-Score) yang kompetitif dan cukup setara dengan `MPLClassifier` dari *library scikit-learn*, asalkan seluruh pengaturan *hyperparameter* internal (seperti *pruning* dan optimasi bawaan) disesuaikan agar pengujian berjalan secara adil (*apple-to-apple*).

3.2 Saran

Untuk pengembangan model dan penelitian lebih lanjut, terdapat beberapa saran untuk dieksplorasi:

1. Eksplorasi Jangkauan Hyperparameter

Kami menyarankan untuk melakukan pencarian *hyperparameter* dengan rentang yang lebih luas, khususnya nilai regularisasi yang lebih cocok dibandingkan 10^{-4} untuk melihat efek penalti bobot dengan lebih jelas.

2. Implementasi Optimasi Lanjutan

Model saat ini bergantung pada *Stochastic Gradient Descent* (SGD), sehingga untuk penelitian lanjutan, disarankan untuk mengimplementasikan *optimizer* tingkat lanjut seperti Adam atau RMSprop untuk mempercepat konvergensi dan meminimalisir *local minimum trap*.

3. Inisialisasi Bobot

Untuk mencegah risiko *vanishing* atau *exploding gradient* sejak tahap awal *training*, kami menyarankan menggunakan teknik inisialisasi bobot seperti *He Initialization* (khusus untuk ReLU), atau *Xavier/Glorot Initialization* untuk Sigmoid/Tanh

4. Cross Validation

Evaluasi model saat ini hanya bergantung pada satu kali proses *train-test split*. Untuk kedepannya, disarankan untuk menerapkan teknik *K-Fold Cross Validation* agar hasil evaluasi metrik lebih baik dan tidak bias terhadap distribusi sampel data tertentu.

REFERENSI

<https://edunex.itb.ac.id/courses/88964/preview/423338>

<https://www.jasonosajima.com/forwardprop>

<https://www.jasonosajima.com/backprop>

<https://numpy.org/doc/2.2/>

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.Html

<https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>

<https://douglasorr.github.io/2021-11-autodiff/article.html>

https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf

<https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/>

<https://builtin.com/data-science/l2-regularization>

<https://builtin.com/data-science/l2-regularization>

[https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions)

LAMPIRAN

Tautan repository GitHub : https://github.com/ahsuunn/Feed-Forward-Neural-Network_ml-1

A. Pembagian Tugas

NIM	Nama	Tugas
13523074	Ahsan Malik Al Farisi	layer.py, normalization.py, activation.py , (Dense Layer, RMSNorm, inisialisasi bobot, fungsi aktivasi)
13523113	Kefas Kurnia Jonathan	autodiff.py (kelas Tensor, operasi forward & backward)
13622076	Ziyan Agil Nur Ramadhan	loss_function.py, optimizer.py, ffn.py (Fungsi loss, Optimizer Adam, kelas utama FFNN, training loop, visualisasi, save/load)

B. Form Penggunaan AI

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : Ahsan Malik Al Farisi / 13523074

Kefas Kurnia Jonathan / 13523113

Ziyan Agil Nur Ramadhan / 13622076

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Fakultas Teknik Mesin dan Dirgantara

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 1 IF3270 Pembelajaran Mesin 2025/2026

Feedforward Neural Network

Menyatakan bahwa ~~saya~~/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya /Tidak*	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya /Tidak*	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/ Tidak *	Sedang
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya /Tidak*	-
Lainnya, Jelaskan: Menggunakan kakas Auto-Latex Equations untuk penulisan persamaan pada laporan.	Ya/ Tidak *	Sedikit

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026

Bandung, 16 Maret 2026

Bandung, 16 Maret 2026



Ahsan Malik Al Farisi

NIM. 13523074



Kefas Kurnia Jonathan

NIM. 13523113



Ziyan Agil Nur Ramadhan

NIM. 13622076